SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 13

Software Requirements Specification (SRS)

Structure, Preparation, Requirement IDs & Traceability

Student Book • Module II: Requirements Engineering

Learning focus
Understand the purpose and structure of an SRS and learn to turn analysed requirements into a clear, organised document that stakeholders, developers, testers, and project managers can use.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Explain the purpose of a Software Requirements Specification (SRS).

• Describe common sections of an SRS and the information each section contains.

• Write uniquely identified, clear, testable functional and non-functional requirements.

• Distinguish requirements from design decisions, assumptions, constraints, and open issues.

• Use traceability to connect stakeholder needs, requirements, and test cases.

• Prepare a small SRS outline for a realistic software system.

> **Core idea:** an SRS is a structured description of what a software system must do and the qualities, interfaces, rules, and constraints it must satisfy. It is a shared reference—not simply a list of features.

2. What Is an SRS?

A **Software Requirements Specification (SRS)** records the agreed requirements for a software product or system. It gives stakeholders a common basis for understanding scope and gives the delivery team a reference for design, implementation, testing, and change control.

Why do teams prepare an SRS?

• To create a shared understanding of the system's scope and expected behaviour.

• To reduce ambiguity and avoid different teams making incompatible assumptions.

• To support planning, estimation, design, implementation, and testing.

• To make review, approval, and change management more systematic.

• To preserve requirement IDs and traceability as the project evolves.

> **Example:** "Build an event app" is not enough for a development team. An SRS clarifies which users can do what, what information is stored, what rules apply, how failures are handled, and what measurable quality expectations must be met.

3. Who Uses the SRS?

Audience	How the SRS helps
Business owner / sponsor	Checks that the system supports the intended goals and scope.
End users / representatives	Confirms that real workflows and expected outcomes are covered.
Developers / designers	Uses requirements to guide system behaviour and design decisions.
Testers / QA team	Derives test conditions and acceptance evidence from requirements.
Project manager	Uses scope and dependencies for planning, estimation, and change control.
Operations / support	Understands service expectations, access rules, and operational constraints.

4. Common SRS Structure

The exact structure varies by organisation, project size, and applicable standard. The following is a practical structure suitable for a college project and many small-to-medium software projects.

Section	What to include
1. Introduction	Purpose of the document, system scope, definitions, abbreviations, references, overview.
2. Overall description	Product context, user classes, operating environment, assumptions, dependencies, and constraints.
3. Functional requirements	System services and behaviours, with unique IDs, conditions, and expected outcomes.
4. External interface requirements	User interfaces, software interfaces, hardware interfaces, and communication interfaces where relevant.

Section	What to include
5. Data requirements	Important entities, fields, validation rules, relationships, retention, and data ownership.
6. Non-functional requirements	Performance, security, reliability, availability, usability, accessibility, maintainability, and other relevant qualities.
7. Business rules and constraints	Policies, eligibility rules, legal obligations, technology constraints, and organisational rules.
8. Models / use cases	Use cases, workflows, context diagrams, or other models that clarify requirements.
9. Acceptance and verification	How the requirements will be checked; acceptance criteria or test references.
10. Appendices / traceability	Glossary, supporting material, requirement source links, change history, and traceability matrix.

Do not add sections merely to make the document longer. Choose a structure that makes the requirements easy to find, review, verify, and maintain.

5. Example SRS — College Event Management System

5.1 Purpose

The College Event Management System (CEMS) is intended to provide a central platform where students can discover published college events and register for events with available capacity. Authorised coordinators can manage event details and review registrations.

5.2 Scope

In scope: event listing, event details, student registration, capacity checks, coordinator event management, role-based permissions, and basic registration reporting.
Out of scope for the initial release: external ticket marketplace integration, online payments, and personalised event recommendations, unless separately approved.

5.3 User classes

User class	Main capabilities
Student	Browse published events, view details, register, view registration status, and follow defined cancellation rules.
Event coordinator	Create and update events, set event capacity, view registrations, and publish events when authorised.
Administrator	Manage user roles and approved system settings; review relevant reports.
IT support	Support account access, system operation, troubleshooting, and approved maintenance tasks.

5.4 Assumptions and constraints

• Users access the system through supported web browsers and an available network connection.

• Role assignments are maintained by an authorised administrator or approved identity service.

• Event eligibility, capacity, and publication rules are supplied or approved by the college.

• Personal information is accessible only to authorised roles and handled according to applicable college policy.

These are example assumptions and constraints. A real project must confirm them with the appropriate stakeholders rather than treating them as automatically true.

6. Writing Functional Requirements

Functional requirements describe observable services and behaviours. Each should have a unique ID and enough detail to avoid competing interpretations.

ID	Requirement statement	Verification idea
FR-01	The system shall display published events to an authenticated student.	Sign in as a student and verify that published events appear.
FR-02	The system shall display the title, date, time, venue, eligibility, and capacity status for each published event.	Inspect event details for each required field.

ID	Requirement statement	Verification idea
FR-03	The system shall allow an eligible student to register for an event when capacity is available.	Register a valid student for an event with an available seat.
FR-04	The system shall reject a registration request when the event has reached its capacity.	Attempt registration for a full event and check the outcome.
FR-05	The system shall permit only authorised coordinator accounts to create or publish events.	Attempt the action with coordinator and student roles.
FR-06	The system shall display the current registration status to the student who owns the registration.	View the status using the owning student account and another account.

Good requirement-writing practices

• Use a unique ID such as FR-01 or NFR-PERF-01.

• Use precise wording and define important terms.

• State conditions and expected outcomes where they matter.

• Avoid combining unrelated behaviours in a single requirement.

• Keep the statement focused on required behaviour rather than unneeded implementation detail.

• Link the requirement to its source, priority, and verification method.

7. Writing Non-Functional Requirements

Non-functional requirements specify measurable qualities or operating constraints. They should be relevant to the system and expressed in a way that allows the team to evaluate them.

ID	Category	Example statement
NFR-PERF-01	Performance	Under the agreed normal-load test, 95% of event-list requests shall complete within two seconds.
NFR-SEC-01	Security	A student-role account shall not be permitted to create or publish an event.
NFR-REL-01	Reliability	A failed registration attempt shall not be stored as a confirmed registration.
NFR-USE-01	Usability	Registration errors shall identify the issue and explain the corrective action in clear language.
NFR-AV-01	Availability	The service shall meet the agreed monthly availability target, excluding defined maintenance windows.

The values and targets above are illustrative. Stakeholders and the delivery team must agree on realistic measures, test conditions, and exceptions for the actual project.

8. Requirements, Design, Assumptions, and Constraints

An SRS should distinguish the need from a proposed solution, while still recording genuine constraints.

Statement	How to treat it
“Students must be able to register for available events.”	Functional requirement: describes required behaviour.
“Use React and Node.js.”	Technology/design decision unless the project has an approved technology constraint.
“All students have college email accounts.”	Assumption until confirmed.
“The system must use the college’s approved identity provider.”	Constraint if imposed by policy or an authorised decision.
“The system should be modern.”	Ambiguous statement that needs clarification.
“The event list shall meet the agreed response-time target.”	Non-functional requirement; define the target and test conditions.

Separating these categories prevents an unconfirmed assumption or personal preference from silently becoming a mandatory system requirement.

9. Requirement IDs, Status, and Priorities

A requirement catalogue helps teams track requirements across reviews and releases. Common fields include:

• Requirement ID

• Requirement statement and type

• Source / stakeholder

• Priority (for example, Must, Should, Could, Won't this time)

• Status (proposed, under review, approved, implemented, verified, deferred, or rejected)

• Dependencies, assumptions, and rationale

• Verification method and related test case

• Version and change history

ID	Type	Priority	Status	Source
FR-01	Functional	Must	Approved	Student interviews
FR-04	Functional	Must	Approved	Coordinator workshop
NFR-PERF-01	Performance	Should / confirm target	Under review	College IT team

10. Traceability and the RTM

Requirements traceability connects a requirement to its source and to related design, implementation, and testing artefacts. A Requirements Traceability Matrix (RTM) is a table commonly used to maintain these relationships.

Requirement ID	Source	Design / component	Test case	Status
FR-03	Student interview	Registration service	TC-REG-01	Approved
FR-04	Coordinator workshop	Capacity validation	TC-CAP-01	Approved
NFR-PERF-01	IT discussion	Event-list API / UI	TC-PERF-01	Under review

Why traceability matters

• Shows where a requirement originated and why it exists.

• Helps verify that every important requirement has an implementation and test plan.

• Reveals which requirements and tests may be affected by a change.

• Supports reviews, audits, acceptance, and impact analysis.

Traceability is not only a final report. Maintain it as requirements are proposed, approved, changed, implemented, and tested.

11. Reviewing and Approving an SRS

An SRS should be reviewed by the people who understand the business need, use the system, build it, and verify it. The review should find issues—not merely collect signatures.

Review checklist

• Does the purpose and scope match the agreed business problem?

• Are relevant user classes and stakeholders represented?

• Are functional requirements clear and sufficiently complete?

• Are important quality attributes, interfaces, data rules, and constraints included?

• Are requirements feasible, consistent, prioritised, and verifiable?

• Are assumptions, dependencies, and open issues visible?

• Does each requirement have a unique ID and source where practical?

• Can testers derive checks or acceptance criteria from the requirements?

• Have unresolved disagreements and excluded features been documented?

Approval

Approval means authorised stakeholders agree that the document is an appropriate baseline for the current scope. It does not mean requirements can never change. After approval, changes should be handled through the agreed change-management process.

12. Change Control After Approval

When a stakeholder requests a change, record it and analyse its effect before modifying the approved baseline.

> Change request → clarify need → impact analysis → assess value, risk, cost, and schedule → approve, defer, or reject → update SRS and version → update traceability and affected plans/tests → communicate the decision.

Example: adding student registration cancellation may affect the registration status model, capacity calculations, coordinator reports, notifications, UI screens, and regression tests. Traceability helps the team discover these impacts.

13. Classroom Activity — Create a Mini SRS

Work in small groups. Prepare a mini SRS for the College Event Management System or another instructor-approved system.

• Write the purpose and scope, including at least two out-of-scope features.

• Identify three user classes and their main goals.

• Write six functional requirements and three non-functional requirements with unique IDs.

• List two assumptions and two constraints, labelling them correctly.

• Add a verification idea for at least five requirements.

• Create an RTM with at least four requirements linked to a source and test case.

• Review another group’s SRS using the checklist and report three strengths and three improvement points.

> **Expected output:** a concise SRS document with requirement IDs, priorities, verification ideas, and a small traceability matrix.

14. Knowledge Check

1. What is the purpose of an SRS?

To document agreed system requirements and constraints in a structured form used by stakeholders and the delivery team.

2. Name four common SRS sections.

Introduction/scope, overall description, functional requirements, non-functional requirements; other sections may cover interfaces, data, models, and traceability.

3. Why should requirements have unique IDs?

To track sources, priorities, status, related tests, and changes without confusion.

4. Is “use React” automatically a functional requirement?

No. It is generally a design or technology choice unless it is an approved constraint.

5. What is an RTM?

A Requirements Traceability Matrix that links requirements to sources and related project artefacts such as design and tests.

6. Can an approved SRS change?

Yes. Changes should be recorded, assessed, authorised, versioned, and reflected in affected artefacts.

7. What makes a non-functional requirement useful?

It describes a relevant quality or constraint with agreed criteria or a practical verification method.

8. Who should review an SRS?

Relevant business stakeholders, users or representatives, developers/designers, testers, and other owners of important constraints.

15. Glossary

Term	Meaning
SRS	Software Requirements Specification; a structured record of system requirements.
Requirement baseline	An approved version used as the reference point for controlled changes.
Requirement ID	A unique identifier for tracking a requirement.
Traceability	The ability to follow a requirement to its source and related artefacts.
RTM	Requirements Traceability Matrix.
Acceptance criteria	Conditions or evidence used to decide whether a requirement is satisfied.
Constraint	A restriction the solution must respect.
Assumption	A belief treated as true but not yet confirmed.
Change control	The process for assessing, approving, recording, and communicating changes.

16. Homework and Deliverables

• Prepare an SRS for CEMS or another approved system using the structure in this chapter.

• Include purpose, scope, user classes, assumptions, constraints, and out-of-scope items.

• Write at least eight functional requirements and four non-functional requirements with unique IDs.

• Assign priorities and statuses to all requirements.

• Add verification ideas to at least six requirements.

• Create an RTM linking at least five requirements to their source and proposed test cases.

• Ask a peer to review your SRS and record three changes made after feedback.

Next session: Requirement validation, feasibility study, and requirement management—how to confirm the SRS is appropriate and control it as the project changes.